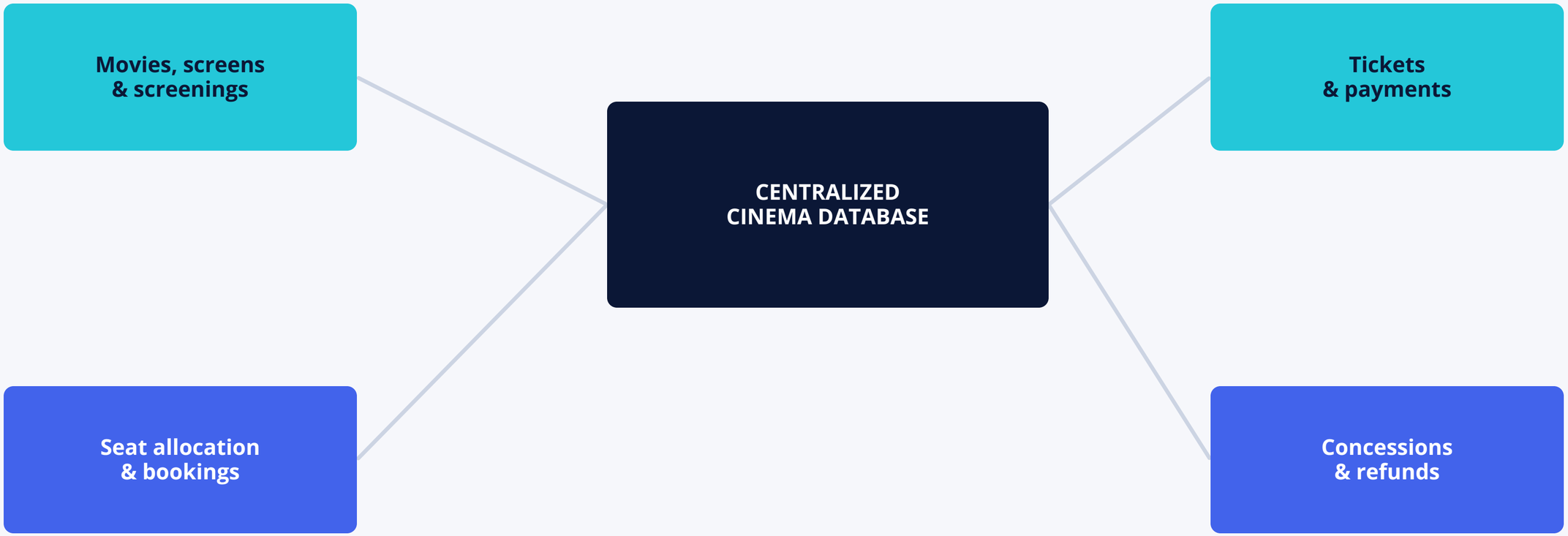
Cinema Screening and Ticket Booking System

DBMS Project – Review 2

Student Name -G.Sahasra Reddy
Roll Number - 25WU0102080
·Section - AIML Rhinos

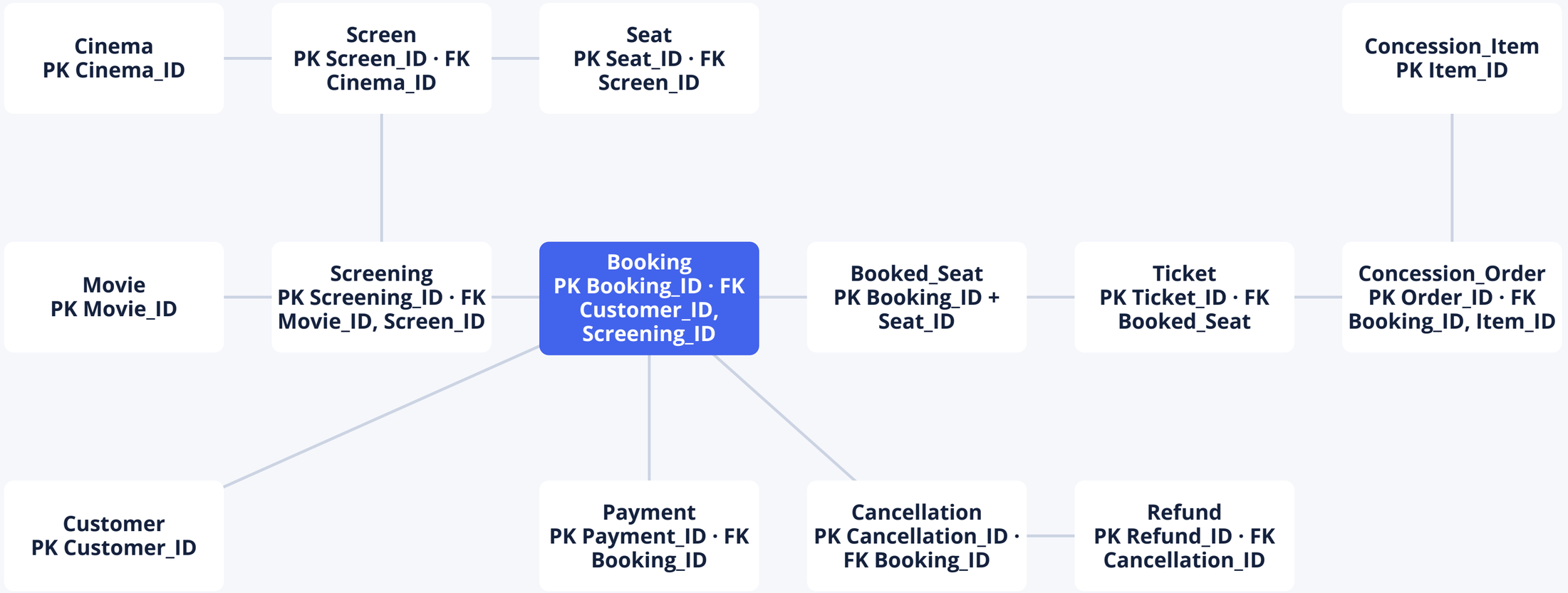


Project Overview



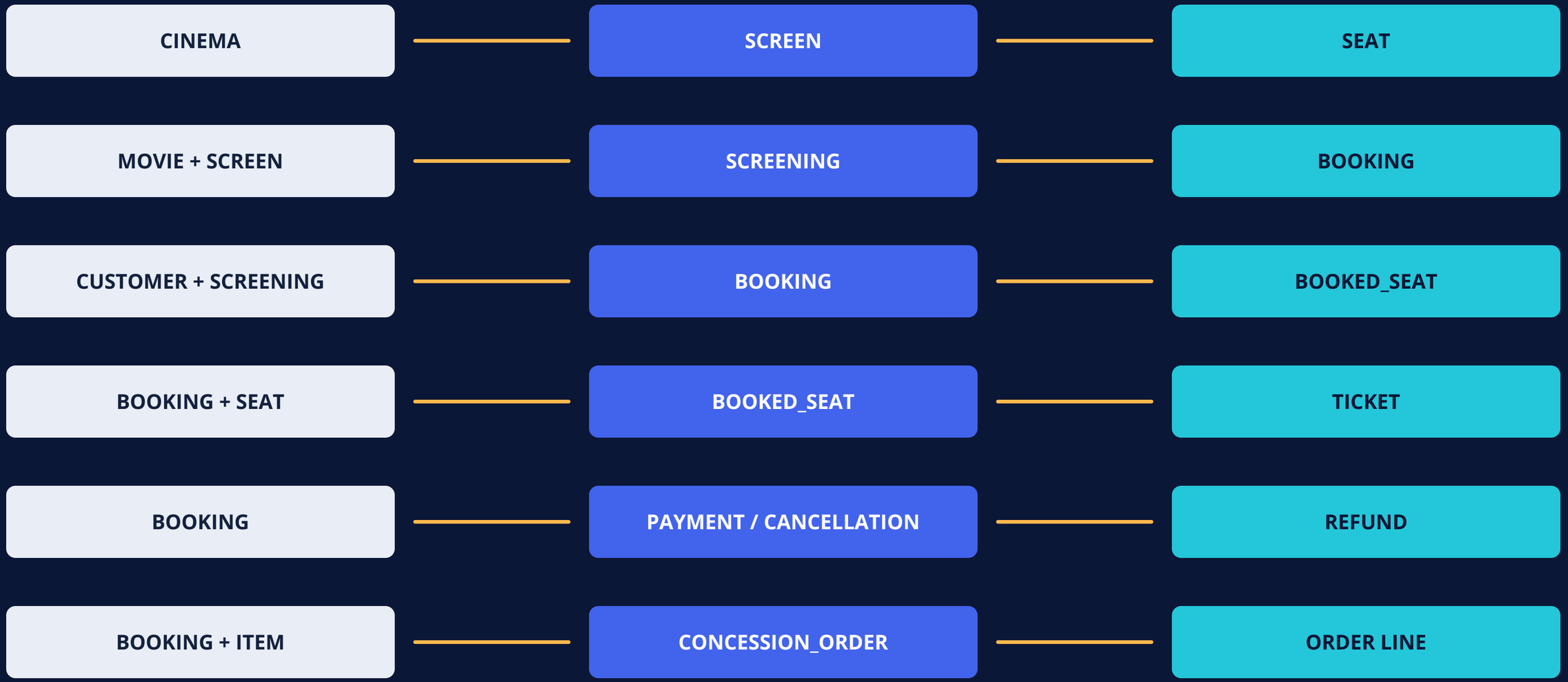
One source of truth for operations from show scheduling to post-booking service.

ER Diagram



1 ——< many PK = Primary Key FK = Foreign Key

Relational Schema



Functional Dependencies

X → Y means X uniquely determines Y

Cinema_ID

Name, Location

Movie_ID

Title, Genre, Duration

Customer_ID

Name, Email, Phone

Screening_ID

Movie_ID, Screen_ID, Start_Time

Booking_ID

Customer_ID, Screening_ID, Status

Item_ID

Item_Name, Price

Normalization up to 3NF

1NF

Atomic values in every attribute

2NF

Remove partial dependencies on composite keys

3NF

Remove transitive dependencies

Distinct CUSTOMER and MOVIE tables • one row per seat allocation • fewer update anomalies

Data Dictionary

TABLE	ATTRIBUTE	DATA TYPE	CONSTRAINT
CINEMA	Cinema_ID	INT (PK)	NOT NULL, UNIQUE
CINEMA	Name / Location	VARCHAR(100)	NOT NULL
SCREEN	Screen_ID / Cinema_ID	INT (PK) / INT (FK)	NOT NULL
SCREEN	Total_Seats	INT	NOT NULL
MOVIE	Movie_ID / Title	INT (PK) / VARCHAR(150)	NOT NULL
MOVIE	Duration	INT (minutes)	NOT NULL
SCREENING	Screening_ID	INT (PK)	NOT NULL
SCREENING	Movie_ID / Screen_ID	INT (FK)	NOT NULL
BOOKING	Booking_ID	INT (PK)	NOT NULL
BOOKING	Customer_ID / Screening_ID	INT (FK)	NOT NULL
PAYMENT	Payment_ID / Booking_ID	INT (PK) / INT (FK)	NOT NULL
PAYMENT	Amount	DECIMAL(10,2)	NOT NULL

DDL and Constraints

PRIMARY KEY
Unique row identifier

FOREIGN KEY
Referential integrity

NOT NULL
Value required

UNIQUE
No duplicates

CHECK
Validate conditions

DEFAULT
Fallback value

```
CREATE TABLE CINEMA (
  Cinema_ID INT PRIMARY KEY,
  Name VARCHAR(100) NOT NULL,
  Location VARCHAR(100) NOT NULL
);
```

Business Rules

01 No duplicate seat per screening

02 No overlapping shows on one screen

03 Concession quantity must be positive

04 Cancellation before deadline

05 Refund only after eligible cancellation

06 Start time must precede end time

Enforced through constraints, transactions and triggers

Sample Data and SQL

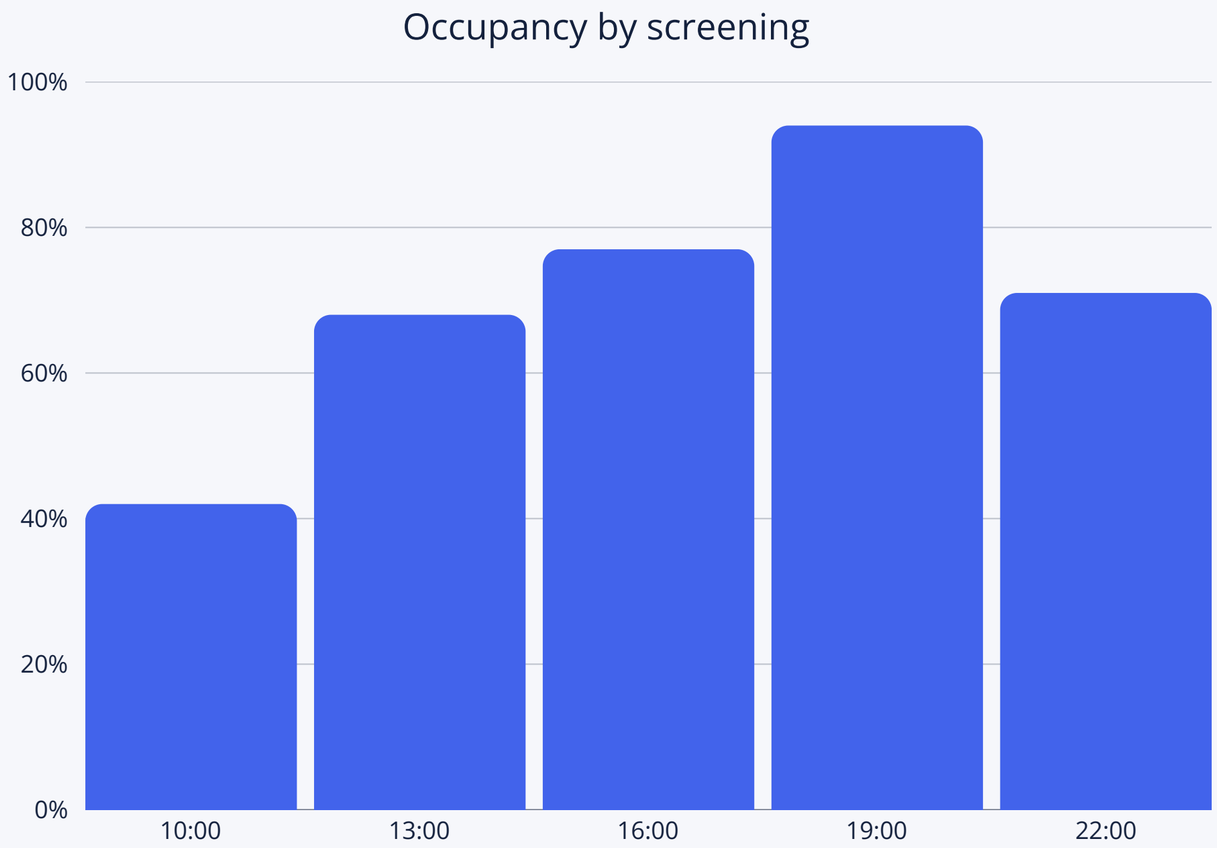
BOOKING DETAILS JOIN

```
SELECT b.Booking_ID, c.Name, m.Title, s.Start_Time
FROM BOOKING b
JOIN CUSTOMER c ON b.Customer_ID = c.Customer_ID
JOIN SCREENING s ON b.Screening_ID = s.Screening_ID
JOIN MOVIE m ON s.Movie_ID = m.Movie_ID;
```

INSERT coverage
Cinemas + screens
Movies + screenings
Customers + bookings

Nested query → customers with > 3 bookings
Aggregation → SUM concession sales · COUNT tickets by movie

Reports, Views and Indexes



Revenue

Movie-wise ticket sales

Utilization

Screen usage rates

Concessions

Sales summaries

Exceptions

Cancellations + refunds

```
CREATE VIEW movie_sales …
```

- `INDEX (Booking_ID, Screening_ID)`

Progress and Conclusion

✓ ER diagram + relational schema	**Completed**
✓ FDs + 3NF normalization	**Completed**
Tables + constraints	**In progress**
Sample data + SQL queries	**In progress**
Reports + prototype demo	**Planned**

Accurate bookings. Stronger data integrity. Better reporting.

A normalized database enables efficient cinema management and more reliable customer service.

Thank You

Questions or feedback?